

German Language Domain — HTML Interface Build Plan v1.0

Date: 2026-05-19

Build target: June 1, 2026

Spec: docs/GERMAN_HTML_SPEC_v1.0.md

Branch: feat/german-domain-extraction (refactor), then feat/german-html-interface (HTML)

Status: Plan approved — awaiting build start authorization

North Star

The German domain reaches v1.0 when it has feature parity with the Curator model:
a primary HTML interface + Telegram as the mobile channel. All domain logic lives
in `german_domain.py`. HTML and Telegram are thin display layers only.

MVP definition: **Lesen + Schreiben + Bibliothek** are functional and usable.

Üben and Admin ship if time permits; they are v1.1 if they don't.

Architecture Decision (non-negotiable)

The refactor comes first. No HTML work begins until `german_domain.py` exists
and all 49 tests pass against it.

Current state: `telegram_bot.py` holds most domain logic.

Target state:

<code>german_domain.py</code>	← all domain logic
■■■ <code>telegram_bot.py</code>	← thin adapter (routing + delivery only)
■■■ <code>html_server.py</code>	← thin adapter (Flask routes + rendering only)

See spec Section 1 for full rationale.

Step 0 — Gate (before any extraction)

Do not skip this. Establish the baseline before touching any code.

1. Run `_NewDomains/language-german/run_tests.py` — confirm all 49 pass
2. Record the exact passing count as the baseline
3. Commit (on feature branch) with message: `test: confirm 49-test baseline before extraction`
4. Every subsequent extraction step must end with all 49 still passing
5. After every extraction step: run the bot manually in `--dry-run` mode and confirm it still responds correctly to a basic "next german session" command before committing

If any test is already failing before extraction starts, stop and fix it first.

Do not carry a broken baseline into the refactor.

Belt-and-suspenders rule (applies to every Group A–D commit):

Automated tests catch logic errors. The dry-run smoke check catches integration breaks that tests might miss — import errors, wiring failures, adapter regressions. Both gates must pass before any extraction step is committed.

Refactor Groups (Week 1, Days 1–3)

Extract in this order. Each group is a discrete commit. Never combine groups in one commit.

Group A — Data types and pure utilities

What: Constants, type definitions, pure functions with no side effects, no async, no Telegram imports.

Candidates:

- Verb conjugation tables
- Phrase scoring logic (fuzzy match thresholds, promotion thresholds)
- Scaffold phrase bank structure
- Session JSON schema definitions
- Persona file parsing utilities

Test gate: All 49 pass after each extraction.

Commit format: refactor(domain): extract Group A – [what]

Group B — Domain logic without async dependencies

What: Functions that hold business logic but don't call `update.message.reply_text` or any other Telegram/async primitive.

Candidates:

- `_resolve_verb()` core logic (verb lookup, fallback)
- `_resolve_phrases()` core logic (phrase selection, filtering)
- Scaffold assembly
- Persona selection logic (slug matching — see Persona Slug note below)
- Session header assembly (see Universal Session Header note below)
- Duration normalization (see Duration note below)

Test gate: All 49 pass after each extraction.

Commit format: refactor(domain): extract Group B – [what]

Group C — Async operations with callback pattern

What: Functions that currently call `update.message.reply_text` mid-execution for progress messages. These must be extracted with the callback pattern so they

stay domain-pure.

Callback pattern (approved):

```
async def _resolve_verb(verb: str, progress_cb: callable | None = None) -> dict:
    if progress_cb:
        await progress_cb("Looking up verb...")
    # domain logic here
```

- Telegram adapter: `progress_cb = lambda msg: update.message.reply_text(msg)`
- HTML adapter: `progress_cb = lambda msg: None` (or SSE push in v1.1)
- Tests: `progress_cb = None`

Candidates:

- `_resolve_verb` (sends "Looking up..." progress)
- `_resolve_phrases` (sends "Generating..." progress)
- `_resolve_drill_verb` (sends progress during LLM call)
- Any other function with mid-execution Telegram sends

Test gate: All 49 pass after each extraction. Add a test that calls each Group C function with `progress_cb=None` to confirm it doesn't raise.

Commit format: `refactor(domain): extract Group C – [what] with callback pattern`

Group D — Wire adapters

What: After all domain logic is in `german_domain.py`, update both adapters to call into it. This is the final refactor step.

`telegram_bot.py` **after Group D:**

- Imports from `german_domain`
- Handles: message routing, voice transcription (Whisper stays here),

Telegram-specific formatting, callback delivery

- Contains no domain logic

`html_server.py` **after Group D:**

- Imports from `german_domain`
- Handles: Flask routes, Jinja2 rendering, Web Speech API bridge
- Contains no domain logic

Test gate: All 49 pass. Bot smoke-test on staging before merging.

Commit format: `refactor: wire telegram adapter to german_domain`

Implementation Notes (from OpenClaw spec review, 2026-05-18)

These apply to the `get_german_session.py` extraction specifically. Address during Group B/C extraction, not deferred to HTML build.

1. Universal Session Header alignment

The `---SESSION---` block format produced by `get_german_session.py` must be identical to what `parse_transcript.py` expects.

Action: Add note to `get_german_session.py`: "Must produce header compatible with Universal Session Header spec (v1.0, Apr 26)." Validate in tests.

2. Duration handling (BUG-014 context)

Enforce "number only" for Duration field or add a normalization helper.

Downstream parsing (`parse_transcript.py`) is fragile to formats like "30 min" vs "30". The `german_domain.py` extraction is the right place to add this guard.

3. Two-bot routing

Both `@minimoi_cmd_bot` and `@minimoi_agent_bot` route to the same Python entrypoint. The domain layer must produce identical output regardless of which bot triggers it. Document this explicitly in `german_domain.py` module docstring.

4. Carry-forward output

If both error counts and vocabulary exist, print on separate lines.

Telegram readability depends on this — a single line becomes unreadable.

5. Persona slug edge cases

Prefer exact filename match before falling back to derived slug.

Example: "Frau Novak" should match `frau_novak.txt` before slug derivation.

Implement in Group B persona selection logic.

6. --dry-run behavior

`get_german_session.py --dry-run` performs all validation and reads but produces no side effects (no file writes, no state mutations).

Document this explicitly. Test: `--dry-run` on a valid session must exit 0 with expected output and leave no files changed.

Flask Server Scaffold (Week 1, Days 4–5)

After the refactor is merged:

1. Create `html_server.py` — Flask app, basic routing
2. Static file serving from `static/`
3. Six-tab navigation skeleton with German names (Lesen, Schreiben, Üben, Bibliothek, Admin, Archiv)
4. No functionality — just the shell with correct structure
5. Aesthetic bones: typography imports (Playfair Display + DM Sans + IBM Plex Mono), colour variables (charcoal, warm gray, terracotta or deep teal — commit to one), CSS custom property skeleton

Tech decisions (confirmed):

- Flask (not FastAPI) — consistency with Curator
- Jinja2 (not React) — sufficient for v1.0; React deferred to Admin tab in v1.1
- Web Speech API (browser-native) — test on target MacBook before committing;

Whisper fallback is a documented one-day addition if unreliable

Week 2 — Feature Tabs (May 26–31)

Lesen tab (Days 6–7)

- RSS fetcher wired to 4–5 seed sources (Heute, Spiegel Panorama, Kicker, ORF)
- Item display: German headline, 2–3 paragraphs, source + date, three actions

(■ ■ ■), pen icon → Schreiben handoff

- Thumbs feedback stored (same learning loop as Curator)
- No scoring personalisation yet — curated list, daily refresh
- **Write and document Lesen scoring prompt in `german_domain.py`** — focused on register and accessibility, not geopolitical signal
- Lesen → Schreiben handoff: pen icon passes article headline + first paragraph

as context to Schreiben tab

Schreiben tab (Days 8–9)

- Freeform text input, no forced prompt
- LLM correction display: original + corrected + notes (gender/case/word order)
- Lesen → Schreiben context pre-fill wired up
- Rotating placeholder text: “*Was hast du heute gesehen?*” and similar Vienna scenes
- Phrase capture: one click from correction view to phrase library
- No voice yet

Voice entry (Day 10)

- Web Speech API microphone button in Schreiben
- Test on target MacBook — Chrome macOS is primary target
- Whisper fallback: Flask endpoint `POST /api/transcribe` if Web Speech unreliable
- Do not deprecate paste flow (Grok Voice on iPhone → paste transcript)

Bibliothek tab (Days 11–12)

- Phrase browser: paginated, filterable (status, scene, verb_hint, date added)
- Inline editing (German/English pairs, verb_hint, status)
- `lessons.json` data structure (Option B — separate file alongside `phrasebook.json`)
- Lesson grouping UI: create lesson, assign phrases, mark active/backlog/archived
- Drill pool visibility: verb card grid with phrase count + last drilled date

Üben tab (Day 13)

- Session launcher: persona + scenario selector, scaffold phrase preview

- Transcript paste flow (same as Telegram paste, but in browser)
- Session results view: error summary, scaffold deployment, Anki cards, carry-forward
- Persona context/memory: rolling 3-session summary injected at session start

(HTML-only if it pushes Telegram prompt over 4096 char limit — see open question 5)

Admin tab + look-and-feel pass (Day 14)

- Persona editor: create + fork (version history is v1.1)
- Typography, colour, texture, German placeholder text
- Mobile layout: single-column below 768px, bottom nav rail
- Touch-friendly tap targets
- Confirm Admin v1.0 scope: CRUD for personas only. Full scenario/scaffold editing is v1.1.

HTML smoke tests: Before v1.0 ships, add basic smoke tests for Lesen feed loading, Schreiben correction handoff, and phrase capture from HTML. Full browser tests are out of scope for v1.0 — basic validation only.

Deferred to v1.1+

Feature	Reason	Target
Lesen personalisation / scoring	Needs thumbs data	v1.1
Auto-dispatch to Grok API	Streaming complexity	v1.1
Error arc chart (Chart.js)	Nice-to-have	v1.1
Archiv tab	Lower frequency	v1.1
Persona version history	Low priority	v1.2
React for Admin tab	Justified at v1.1 form complexity	v1.1
Neo4j cross-session intelligence	Requires v1.3 infra	v1.3

Open Technical Questions

Answers needed before or during build — not blocking spec approval.

1. **Flask vs FastAPI** — Flask confirmed for v1.0 (consistency with Curator)
2. **Jinja2 vs React** — Jinja2 for v1.0; React for Admin in v1.1
3. **Web Speech API reliability** — test on target MacBook before committing
4. **Prompt assembly for HTML sessions** — one assembler with channel parameter or two paths? Decide before building Üben session launcher. Telegram limit: 4096 chars.
5. **Persona memory prompt size** — test with Georg + Maria at max assembled size before committing to memory injection in both channels

Scope Safety Valve

If the refactor runs long (>3 days), cut scope in this order:

1. Üben → v1.1
2. Admin → v1.1
3. Ship: Lesen + Schreiben + Bibliothek (the reading-writing loop)

Do not ship a broken refactor to make the June 1 date.

A clean `german_domain.py` with Lesen + Schreiben is a better v1.0 than a rushed extraction with HTML bolted on top of tangled state.

Mac Mini Deployment Checklist

- nginx config for Flask app (reverse proxy + static serving)
- HTTPS setup (required for Web Speech API to work in browser)
- launchd plist for `html_server.py`
- Confirm `html_server` added to `health_monitor` service list
- Test Web Speech API on target MacBook (Chrome macOS primary)

Post-Build Checklist (for OpenClaw after merge)

- Update `PLAN.md` with HTML interface milestone status
- Update `CHECKLIST.md` — check off refactor and HTML build items
- Reference this spec in German domain roadmap
- Update `GERMAN.md` status: `v0.9 beta` → `v1.0 in progress`
- Update `OPERATIONS_ROADMAP.md`: HTML interface as v1.0 milestone
- File GitHub issue: `feat: German domain HTML interface v1.0 — design approved`

Labels: enhancement, language-domain, v1.0-milestone, Milestone: June 1

German Language Domain — HTML Interface Build Plan v1.0

2026-05-19 — based on `GERMAN_HTML_SPEC_v1.0.md` + OpenClaw review 2026-05-18